

INFORME TÉCNICO DE ARQUITECTURA Y TRANSACCIONALIDAD DEL SOFTWARE

*Diseño, Implementación Transaccional Robusta y Control de Concurrencia con Clean Architecture,
WPF (MVVM) y ADO.NET sobre Microsoft SQL Server*

**Alvarado Minchan Cristian Paul | Chicoma García Álvaro Jesús | Méndez Esparza Carlos Andrés | Saldaña
Vargas Nikolas Faviano**

*Facultad de Ingeniería, Escuela Académico Profesional de Ingeniería de Sistemas
Universidad Nacional de Cajamarca, Cajamarca, Perú*

I. RESUMEN EJECUTIVO (ABSTRACT)

Resumen Ejecutivo (Abstract)— El presente informe técnico expone el diseño e implementación del Sistema Integrado de Gestión de Transportes y Encomiendas Interprovinciales, una solución de software de escritorio concebida para automatizar de manera unificada la emisión de boletos de viaje y la administración logística de carga liviana. La problemática abordada comprende la erradicación de condiciones de carrera y sobreventa de butacas por colisión concurrente entre terminales remotas, así como la eliminación de descuadres contables derivados de fallos transaccionales intermedios en ventanilla. La solución adopta los principios de Clean Architecture, segregando formalmente las capas de Dominio, Aplicación, Infraestructura y Presentación, desacopladas mediante el patrón Modelo-Vista-Modelo de Vista (MVVM) sobre Windows Presentation Foundation (WPF) en plataforma .NET 10. La persistencia se fundamenta en ADO.NET nativo (Microsoft.Data.SqlClient) gobernado mediante SqlTransaction explícito, descartando mapeadores objeto-relacionales para maximizar el rendimiento y garantizar aislamiento transaccional granular. Entre los principales entregables destacan un croquis interactivo de cuarenta butacas en dos niveles con pasillo central, despacho multimodal de bodega, recaudación multicanal, precalentamiento asíncrono del pool de conexiones bajo cifrado TLS y un motor de concurrencia atómica basado en verificación condicional de estado. La evaluación experimental valida que ante colisiones simultáneas en la selección de butacas, el sistema ejecuta una reversión transaccional completa (Rollback), preservando la invariabilidad de la caja y garantizando integridad ACID absoluta sin estados huérfanos ni inconsistencias operativas.

Palabras Clave (Index Terms)— Clean Architecture, MVVM, Windows Presentation Foundation (WPF), ADO.NET, SqlTransaction, Control de Concurrencia Atómica, Integridad Transaccional ACID.

II. DESCRIPCIÓN DEL DOMINIO Y PROBLEMA

Modelo de Negocio y Proceso Operativo— El sistema de software desarrollado en la solución de Visual Studio ('New Proyect') sistematiza el modelo operativo y comercial de una empresa de transporte terrestre interprovincial y paquetería de bodega. La plataforma automatiza de manera integrada la programación de itinerarios de salida en corredores regionales del norte y centro del país (tales como Cajamarca, Chepén, Chiclayo, Trujillo y Lima), la comercialización de butacas numeradas en una flota vehicular de buses de dos niveles con cuarenta asientos homologados (doce en nivel VIP y veintiocho en nivel Ejecutivo), la recepción y despacho de encomiendas y equipaje con sobrepeso en bodega mediante pesaje en báscula industrial con límite operativo de 50 kg, y el control financiero de los turnos de caja asignados a los operadores de ventanilla.

Problemática Operativa y Necesidad Transaccional— En el entorno operativo cotidiano, múltiples operadores de ventanilla en agencias remotas consultan y procesan transacciones de forma concurrente sobre los mismos servicios de transporte. La carencia de un gobierno transaccional explícito genera tres patologías críticas que comprometen la viabilidad operativa y financiera de la empresa: a) Sobreventa de butacas por condiciones de carrera (Race Conditions), donde dos terminales leen simultáneamente un asiento en estado libre y confirman su venta en paralelo, produciendo boletos duplicados, conflictos con los usuarios y severas sanciones administrativas impuestas por el organismo regulador SUTRAN; b) Descuadres monetarios y ruptura del balance de caja, derivados de interrupciones de red o fallos a la mitad de la secuencia de compra, lo que origina boletos emitidos sin registro contable de cobro o fondos acumulados en caja sin pasajes de respaldo; y c) Asientos huérfanos bloqueados indefinidamente por operaciones abortadas sin reversión automática.

Justificación de la Solución Arquitectónica— Frente a estas contingencias, surge la imperiosa necesidad de diseñar e implementar una arquitectura de persistencia transaccional basada en el estándar ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad). Mediante el empleo de transacciones relacionales explícitas gobernadas por ADO.NET (SqlTransaction) y bloqueos de granularidad fina a nivel de fila (UPDLOCK, ROWLOCK), el sistema garantiza que la reserva de la butaca, la emisión del comprobante, el registro de la guía de encomienda y la actualización del saldo de la caja se consoliden bajo el principio de indivisibilidad: o se confirman todas las operaciones con éxito absoluto (Commit), o se revierte la totalidad de las modificaciones previas (Rollback), preservando la integridad del inventario y la concordancia contable.

III. ALCANCE Y REQUERIMIENTOS FUNCIONALES

A. Módulos Desarrollados en la Solución

La inspección técnica de la solución de software permite delimitar su alcance en los siguientes ocho módulos funcionales, redactados en formato de lista directa de características según la guía oficial:

- **Módulo de Seguridad y Autenticación:** Permite validar credenciales de operadores y administradores contrastando resúmenes criptográficos SHA-256 en base de datos, restringir accesos según roles y ejecutar el precalentamiento asíncrono del pool de conexiones.
- **Módulo de Gestión y Arqueo de Turnos de Caja:** Permite aperturar turnos con saldo inicial de sencillo (S/. 500.00), supervisar el balance vivo en tiempo real y bloquear cualquier intento de emisión si el operador carece de una caja activa.
- **Módulo de Catálogo y Búsqueda Dinámica de Viajes:** Permite filtrar itinerarios por origen y destino en cascada, acotar fechas de salida hasta 60 días futuros y de retorno hasta 30 días, aplicar descuentos comerciales por viaje de ida y vuelta (+75% del valor base) e inhabilitar salidas expiradas.
- **Módulo de Croquis Interactivo de Asientos:** Permite renderizar la topología física real del bus de dos niveles (40 asientos con pasillo central divisor), gestionar una máquina de estados visual reactiva (Libre, Seleccionado y Ocupado) y restringir la compra a un tope máximo de 5 butacas por transacción.

- **Módulo de Pasajeros y Liquidación Financiera:** Permite capturar nominalmente los datos de los viajeros validando DNI de 8 dígitos numéricos y nombres obligatorios, calcular el vuelto en tiempo real para cobros en Efectivo y validar referencias de pago digital en Yape, Plin y Tarjetas.
- **Módulo de Encomiendas y Logística de Bodega:** Permite liquidar sobrepeso vinculado a boletos mediante tarifas escalonadas (presets de S/. 10.00 a S/. 50.00) y emitir guías independientes de paquetería de hasta 50 kg con datos de remitente y destinatario, admitiendo entrega en agencia o a domicilio con recargo.
- **Módulo de Control de Integridad y Concurrencia ACID:** Permite verificar y bloquear asientos mediante sentencias atómicas y cerrojos de fila (UPDLOCK, ROWLOCK), gobernar la transacción relacional explícita y ejecutar el Rollback automático e integral ante colisiones concurrentes.
- **Módulo de Auditoría de Operaciones y Trazabilidad:** Permite consultar en caliente el histórico de boletos y encomiendas despachadas por turno de caja, facilitando la conciliación contable y la detección de discrepancias operativas.

B. Detalle de Características Funcionales Técnicas Directas

Las especificaciones técnicas directas implementadas en el sistema se desglosan a continuación bajo el estándar de la guía:

- **Módulo de Seguridad:** Permite validar contraseñas cifradas en SHA-256 almacenadas en el campo PasswordHash de dbo.Usuarios.
- **Módulo de Seguridad:** Permite ejecutar una rutina de precalentamiento asíncrono (SELECT 1;) al inicializar el Login para eliminar la latencia de conexión.
- **Módulo de Turnos de Caja:** Permite verificar automáticamente al iniciar sesión si el operador dispone de una caja en estado 'Abierta' en dbo.CajasTurno.
- **Módulo de Turnos de Caja:** Permite visualizar y actualizar en tiempo real el saldo de caja tras cada transacción mediante enlace reactivo con la UI.
- **Módulo de Búsqueda de Viajes:** Permite consultar dinámicamente los orígenes y cargar en cascada únicamente los destinos con salidas programadas vigentes.
- **Módulo de Búsqueda de Viajes:** Permite restringir la selección de fecha de viaje entre la fecha actual del sistema y un límite superior de 60 días calendario.
- **Módulo de Búsqueda de Viajes:** Permite calcular una tarifa promocional automática al seleccionar itinerarios de ida y vuelta (+75% del costo base del tramo).
- **Módulo de Búsqueda de Viajes:** Permite inhabilitar visual y lógicamente aquellos viajes cuya hora de salida programada haya expirado respecto al reloj del servidor.
- **Módulo de Croquis de Asientos:** Permite representar de forma fiel la nave del bus de dos pisos, distribuyendo 12 asientos VIP en el Piso 1 y 28 asientos en el Piso 2.
- **Módulo de Croquis de Asientos:** Permite actualizar reactivamente el color del asiento según su estado: Verde Esmeralda (Libre), Azul Cobalto (Seleccionado) y Rojo (Ocupado).
- **Módulo de Croquis de Asientos:** Permite bloquear la selección de asientos cuando el usuario alcanza el límite de compra de 5 butacas en la misma operación.
- **Módulo de Pasajeros:** Permite validar estrictamente que el Documento Nacional de Identidad (DNI) contenga exactamente 8 caracteres numéricos mediante expresiones regulares.
- **Módulo de Pasajeros:** Permite condicionar la habilitación del botón de pago al ingreso obligatorio de los nombres completos de cada pasajero.
- **Módulo de Liquidación:** Permite calcular de forma instantánea el cambio o vuelto cuando el cliente abona en Efectivo, validando que el monto cubra el total.
- **Módulo de Liquidación:** Permite verificar códigos de operación digital de 8 dígitos para Yape / Plin y referencias alfanuméricas de voucher para tarjetas bancarias.
- **Módulo de Carga y Encomienda:** Permite seleccionar tarifas rápidas de sobrepeso para pasajeros con tarifas preconfiguradas de S/. 10.00, S/. 20.00, S/. 30.00 y S/. 50.00.

- **Módulo de Módulo de Carga y Encomienda:** Permite tramitar encomiendas puras de bodega sin exigir la compra de un pasaje de viaje asociado.
- **Módulo de Módulo de Carga y Encomienda:** Permite validar que el peso registrado en báscula sea estrictamente mayor a 0 kg y no supere la capacidad máxima de 50 kg por paquete.
- **Módulo de Módulo de Carga y Encomienda:** Permite aplicar un recargo logístico reglamentario de S/. 10.00 cuando se elige entrega a domicilio, exigiendo el ingreso de dirección.
- **Módulo de Módulo de Integridad ACID:** Permite adquirir bloqueos de fila exclusivos (WITH UPDLOCK, ROWLOCK) al momento de reservar cada asiento en dbo.EstadoAsientosViaje.
- **Módulo de Módulo de Integridad ACID:** Permite garantizar la unicidad de butacas vendidas a nivel de base de datos mediante el índice UQ_Boletos_Viaje_Asiento.
- **Módulo de Módulo de Integridad ACID:** Permite invocar el método RollbackAsync() de SqlTransaction cancelando todas las sentencias DML si surge cualquier error en la cadena.
- **Módulo de Módulo de Integridad ACID:** Permite notificar al operador mediante diálogo modal y refrescar el croquis en caliente si otro terminal ganó la butaca milisegundos antes.
- **Módulo de Módulo de Auditoría:** Permite listar, filtrar y exportar los comprobantes de boletos y encomiendas emitidos durante el turno operativo para el cuadro de caja.

C. Tabla Resumen de Requerimientos Funcionales y Trazabilidad

La Tabla I resume los requerimientos funcionales técnicos del sistema junto con su mapeo hacia las capas de software y artefactos de persistencia:

TABLA I: MATRIZ DE TRAZABILIDAD: REQUERIMIENTOS FUNCIONALES VS. COMPONENTES Y BASE DE DATOS

ID	Requerimiento Funcional	Módulo	Capa	Componente C# / ViewModel	Artefacto de BD
RF-01	Autenticación Criptográfica SHA-256	Seguridad	Application / Infra	AuthService, SecurityService, UsuarioRepo	dbo.Usuarios, HASHBYTES
RF-02	Precalentamiento Asíncrono Pool	Seguridad	UI / Infra	App.xaml.cs, SqlConnectionFactory	master/BD_Transportes (SELECT 1)
RF-03	Auto-Vinculación de Turno de Caja	Turnos Caja	Application / Infra	AuthService, CajaTurnoRepository	dbo.CajasTurno (Estado='Abierta')
RF-04	Monitoreo Reactivo Saldo de Caja	Turnos Caja	UI (WPF)	VentaIntegradaViewModel.SaldoCajaActual	dbo.CajasTurno (MontoActual)
RF-05	Filtrado en Cascada Origen-Destino	Búsqueda Viajes	Application / Infra	ConsultaViajesService, ViajeRepository	dbo.Viajes (IX_Viajes_Origen_Destino)
RF-06	Acotamiento Fecha Salida (60 días)	Búsqueda Viajes	UI / Application	VentaIntegradaViewModel.FechaMaximaSalida	dbo.Viajes (FechaSalida)
RF-07	Tarificación Promocional Ida/Vuelta	Búsqueda Viajes	Application / UI	ConsultaViajesService, VentaIntegradaVM	dbo.Viajes (PrecioBase * 1.75)
RF-08	Filtrado de Viajes Expirados	Búsqueda Viajes	Application / UI	ViajeItemViewModel.SalidaDisponible	dbo.Viajes (FechaSalida > GETDATE())
RF-09	Croquis Topológico Bus 2 Pisos	Croquis Asientos	UI (WPF)	VentaBoletosWindow.xaml, AsientoItemVM	dbo.Asientos, dbo.Buses

ID	Requerimiento Funcional	Módulo	Capa	Componente C# / ViewModel	Artefacto de BD
RF-10	Máquina de Estados Cromática	Croquis Asientos	UI (WPF)	EstadoAsientoToBrushConverter, AsientoVM	dbo.EstadoAsientos Viaje (Estado)
RF-11	Límite Máximo 5 Butacas / Venta	Croquis Asientos	Application / UI	VentaService, VentaIntegradaViewModel	Validación en memoria (Count <= 5)
RF-12	Validación DNI Pasajero (8 dígitos)	Pasajeros	Application	VentaService.DniRegex, PasajeroItemVM	dbo.Boletos (CHK_Boletos_DniPasajero)
RF-13	Nombres Completos Obligatorios	Pasajeros	Application / UI	VentaService, PasajeroItemViewModel	dbo.Boletos (NombrePasajero NOT NULL)
RF-14	Gestión de Efectivo y Vuelto	Liquidación	UI (WPF)	VentaIntegradaViewModel.Vuelto	dbo.CajasTurno (MontoActual)
RF-15	Validación de Pagos Digitales	Liquidación	Application / UI	VentaService, VentaIntegradaViewModel	dbo.Boletos (MetodoPago, NumOperacion)
RF-16	Presets Equipaje con Sobrepeso	Carga/Bodega	UI / Application	PresetItemViewModel, VentaService	dbo.Encomiendas (CostoCarga)
RF-17	Despacho Encomienda sin Boleto	Carga/Bodega	Application / Infra	VentaService, VentaRepository	dbo.Encomiendas (BoletoID IS NULL)
RF-18	Límite Capacidad Carga (50 kg)	Carga/Bodega	Application	VentaService, EncomiendaDTO	dbo.Encomiendas (CHK_Encomiendas_PesoKg)
RF-19	Recargo Delivery Domicilio (S/.10)	Carga/Bodega	Application / UI	VentaIntegradaViewModel.RecargoDelivery	dbo.Encomiendas (RecargoDelivery)
RF-20	Bloqueo de Concurrencia de Fila	Integridad ACID	Infraestructura	VentaRepository (UPDLOCK, ROWLOCK)	dbo.EstadoAsientos Viaje
RF-21	Restricción Unicidad Boleto	Integridad ACID	Infraestructura / BD	VentaRepository (captura SqlEx 2601/2627)	UQ_Boletos_Viaje_Asiento
RF-22	Reversión Atómica Total Rollback	Integridad ACID	Infraestructura	VentaRepository (SqlTransaction.Rollback)	Transacción SQL Server (@@TRANCOUNT)
RF-23	Sincronización Reactiva Croquis	Integridad ACID	UI (WPF)	VentaIntegradaViewModel.ConsultarAsientos	dbo.EstadoAsientos Viaje (Estado)
RF-24	Auditoría de Turnos y Trazabilidad	Auditoría	Application / Infra	AuditoriaService, HistorialRepository	dbo.Boletos, dbo.Encomiendas, CajasTurno

IV. ARQUITECTURA DEL SOFTWARE (CLEAN ARCHITECTURE + MVVM)

La solución 'New Proyect' implementa una arquitectura desacoplada basada en Clean Architecture (Arquitectura Limpia) y el patrón de presentación MVVM puro sobre Windows Presentation Foundation (WPF) en plataforma .NET 10. La meta primordial de este diseño es garantizar la estricta separación de responsabilidades y la inversión de dependencias, aislando las reglas de negocio del framework y los detalles de infraestructura de persistencia.

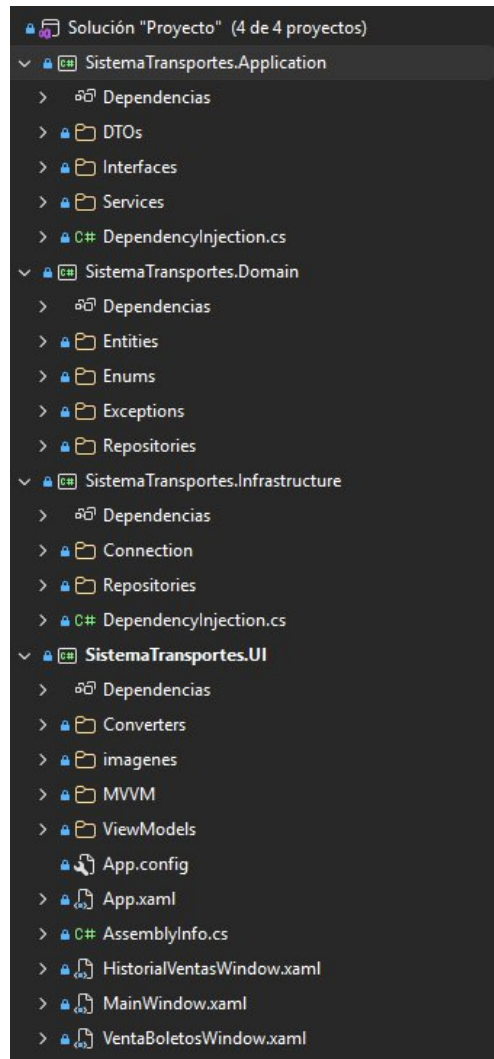


Fig. 1. Estructura de proyectos y segregación modular de capas en Visual Studio Solution Explorer.

A. Responsabilidad por Capas de Software

1. Capa de Dominio (SistemaTransportes.Domain)

Constituye el núcleo agnóstico del sistema, exento de dependencias externas. Contiene:

- **Entidades de Negocio (Entities):** Clases puras POCO que encapsulan el estado y las reglas intrínsecas: Usuario, CajaTurno, Bus, Asiento, Viaje, EstadoAsientoViaje, Boleto y Encomienda.
- **Enumeradores Oficiales:** Tipos discretos fuertemente tipados: EstadoAsiento, EstadoCaja, MetodoPagoEnum y ModalidadEntrega.

- **Excepciones de Dominio Tipadas:** Jerarquía de errores del dominio: AsientoNoDisponibleException, CajaNoActivaException, EntidadNoEncontradaException y VentaValidationException.
- **Interfaces de Repositorio (IRepository):** Contratos de persistencia que materializan el principio de Inversión de Dependencias (DIP): IVentaRepository, ICajaTurnoRepository, IViajeRepository, IAsientoRepository, IHistorialRepository e IUsuarioRepository.

2. Capa de Aplicación (SistemaTransportes.Application)

Orquesta los flujos de trabajo y casos de uso del sistema dependiendo exclusivamente del Dominio. Comprende:

- **Servicios de Aplicación:** Servicios que coordinan la lógica del negocio: VentaService, ConsultaViajesService, AuthService, AuditoriaService y SecurityService.
- **Objetos de Transferencia de Datos (DTOs):** Estructuras planas para el transporte seguro de datos sin acoplar la UI: RegistroVentaRequestDTO, PasajeroDTO, EncomiendaDTO, ViajeDTO y AuditoriaTurnoDTO.
- **Validaciones de Caso de Uso:** Validación estricta previa a la persistencia mediante expresiones regulares (DNI de 8 dígitos, teléfonos) y restricciones de capacidad (límite de 5 asientos y 50 kg en bodega).

3. Capa de Infraestructura (SistemaTransportes.Infrastructure)

Implementa materialmente los contratos prescritos por el Dominio utilizando ADO.NET nativo (Microsoft.Data.SqlClient). Se descartó el uso de ORMs pesados (como Entity Framework Core) para evitar penalizaciones de latencia y permitir un control exhaustivo y quirúrgico de bloqueos pesimistas de fila (UPDLOCK, ROWLOCK) y del ciclo de vida de SqlTransaction.

4. Capa de Presentación y UI (SistemaTransportes.UI)

Construida en WPF bajo MVVM puro sin lógica de negocio en el code-behind (.xaml.cs). Configura el contenedor IoC (Microsoft.Extensions.DependencyInjection) en App.xaml.cs y define ventanas XAML, ViewModels y ValueConverters.

B. Implementación de MVVM y Mecanismos de Desacoplamiento

El desacoplamiento entre la vista y la lógica de presentación descansa en los siguientes tres mecanismos:

1. Notificación de Cambio de Estado: INotifyPropertyChanged y OnPropertyChanged

La clase abstracta ViewModelBase encapsula la invocación de PropertyChangedEventHandler mediante [CallerMemberName], garantizando que toda mutación en las propiedades privadas del ViewModel refresque reactivamente los controles en pantalla:

Listado 1. Clase base ViewModelBase implementando INotifyPropertyChanged con SetProperty genérico.

```
// SistemaTransportes.UI/MVVM/ViewModelBase.cs
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace SistemaTransportes.UI.MVVM;

public abstract class ViewModelBase : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler? PropertyChanged;

    protected virtual void OnPropertyChanged([CallerMemberName] string? propertyName = null)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }
}
```

```

protected bool SetProperty<T>(ref T storage, T value, [CallerMemberName] string? propertyName = null)
{
    if (EqualityComparer<T>.Default.Equals(storage, value))
        return false;

    storage = value;
    OnPropertyChanged(propertyName);
    return true;
}
}

```

2. Colecciones Reactivas: *ObservableCollection<T>* frente a *List<T>*

El uso de *List<T>* carece de eventos de notificación, impidiendo que el motor de WPF se entere cuando se agregan o eliminan elementos. En cambio, *ObservableCollection<T>* implementa *INotifyCollectionChanged*. Cuando se añade un pasajero o se conmuta una fila de asientos, se dispara el evento *CollectionChanged*, refrescando automáticamente los controles *ItemsControl* y *DataGrid* sin reconstruir el árbol visual.

3. Patrón Command: Implementación de *ICommand* y *RelayCommand*

Para desacoplar los eventos de interfaz (*Button.Click*) de la lógica en C#, se implementó *RelayCommand*, encapsulando delegados *Action* (*Execute*) y *Predicate* (*CanExecute*) enlazados a *CommandManager.RequerySuggested*:

Listado 2. Implementación del Patrón Command mediante *RelayCommand* desacoplado.

```

// SistemaTransportes.UI/MVVM/RelayCommand.cs
using System.Windows.Input;

namespace SistemaTransportes.UI.MVVM;

public class RelayCommand : ICommand
{
    private readonly Action<object?> _execute;
    private readonly Predicate<object?>? _canExecute;

    public RelayCommand(Action<object?> execute, Predicate<object?>? canExecute = null)
    {
        _execute = execute ?? throw new ArgumentNullException(nameof(execute));
        _canExecute = canExecute;
    }

    public RelayCommand(Action execute, Func<bool>? canExecute = null)
        : this(_ => execute(), canExecute == null ? null : _ => canExecute()) { }

    public event EventHandler? CanExecuteChanged
    {
        add => CommandManager.RequerySuggested += value;
        remove => CommandManager.RequerySuggested -= value;
    }
}

```

```

public bool CanExecute(object? parameter) => _canExecute == null || _canExecute(parameter);
public void Execute(object? parameter) => _execute(parameter);
public void RaiseCanExecuteChanged() => CommandManager.InvalidateRequerySuggested();
}

```

4. Fragmento de Código C# Real del ViewModel (VentaIntegradaViewModel.cs)

El siguiente fragmento de VentaIntegradaViewModel.cs evidencia la gestión de propiedades reactivas, colecciones y el comando asíncrono para confirmar la venta:

Listado 3. Orquestación reactiva y manejo de comandos en VentaIntegradaViewModel.cs.

```

// Fragmento representativo de VentaIntegradaViewModel.cs
public class VentaIntegradaViewModel : ViewModelBase
{
    private readonly IVentaService _ventaService;
    private readonly ICajaTurnoRepository _cajaTurnoRepository;

    private decimal _saldoCajaActual;
    public decimal SaldoCajaActual
    {
        get => _saldoCajaActual;
        set => SetProperty(ref _saldoCajaActual, value);
    }

    public ObservableCollection<PasajeroItemViewModel> Pasajeros { get; } = new();
    public ObservableCollection<AsientoItemViewModel> AsientosSeleccionados { get; } = new();

    public ICommand ConfirmarVentaFinalCommand { get; }

    public VentaIntegradaViewModel(IVentaService ventaService, ICajaTurnoRepository cajaTurnoRepo)
    {
        _ventaService = ventaService;
        _cajaTurnoRepository = cajaTurnoRepo;
        ConfirmarVentaFinalCommand = new RelayCommand(async () => await ConfirmarVentaAsync());
    }

    public async Task ConfirmarVentaAsync()
    {
        var request = new RegistroVentaRequestDTO { /* ... mapeo de DTO ... */ };
        try
        {
            await _ventaService.RegistrarVentaAsync(request);
            SaldoCajaActual = await _cajaTurnoRepository.ObtenerSaldoActualAsync(Session.CajaTurnoID);
            NotificarMensaje?.Invoke("¡Venta confirmada exitosamente!", "Éxito", MessageBoxImage.Information);
        }
        catch (AsientoNoDisponibleException ex)
        {
            await ConsultarAsientosAsync(ViajeSeleccionado.ViajeID); // Refresco en caliente
        }
    }
}

```

```

        NotificarMensaje?.Invoke(ex.Message, "Conflicto Concurrente", MessageBoxImage.Warning);
    }
}
}

```

5. Fragmento de Código XAML Real de la Vista (VentaBoletosWindow.xaml)

El siguiente fragmento extraído de VentaBoletosWindow.xaml ilustra cómo la interfaz se enlaza a los comandos y propiedades del ViewModel mediante Data Binding, sin código en el code-behind (.xaml.cs):

Listado 4. Declaración XAML desacoplada con Data Binding, DataTriggers y Command Binding.

```

<!-- Fragmento de VentaBoletosWindow.xaml: Plantilla interactiva de butacas -->
<Style x:Key="BotonAsientoStyle" TargetType="Button">
    <Setter Property="Command" Value="{Binding ClickCommand}" />
    <Setter Property="IsEnabled" Value="{Binding EsClickeable}" />
    <Setter Property="Height" Value="46" />
    <Setter Property="Template">
        <Setter.Value>
            <ControlTemplate TargetType="Button">
                <Border x:Name="b" Background="{Binding EstadoVisual, Converter={StaticResource
EstadoAsientoConverter}}"
                    CornerRadius="6" Padding="4,2">
                    <StackPanel VerticalAlignment="Center">
                        <TextBlock Text="{Binding NroAsiento, StringFormat='N° {0}'}" FontWeight="Bold"
Foreground="White" />
                        <TextBlock Text="{Binding Estado}" FontSize="8.5" Foreground="#F1F5F9" />
                    </StackPanel>
                </Border>
                <ControlTemplate.Triggers>
                    <DataTrigger Binding="{Binding EsSeleccionado}" Value="True">
                        <Setter TargetName="b" Property="BorderBrush" Value="#FF6B00" />
                        <Setter TargetName="b" Property="BorderThickness" Value="2.5" />
                    </DataTrigger>
                </ControlTemplate.Triggers>
            </ControlTemplate>
        </Setter.Value>
    </Setter>
</Style>

<!-- Botón de Confirmación desacoplado enlazado mediante Command -->
<Button Content="Confirmar y Emitir Venta"
    Command="{Binding ConfirmarVentaFinalCommand}"
    IsEnabled="{Binding PuedeConfirmarVenta}"
    Height="44" Margin="0,10,0,0" Style="{StaticResource BtnPrimaryStyle}" />

```

V. DISEÑO DE BASE DE DATOS Y PERSISTENCIA ADO.NET

La persistencia del sistema descansa sobre el motor relacional Microsoft SQL Server (compatible con Azure SQL Database). El modelo relacional se concibió bajo la Tercera Forma Normal (3FN), garantizando la atomicidad, eliminando dependencias transitivas y optimizando el acceso mediante índices agrupados y no agrupados con cobertura (INCLUDE).

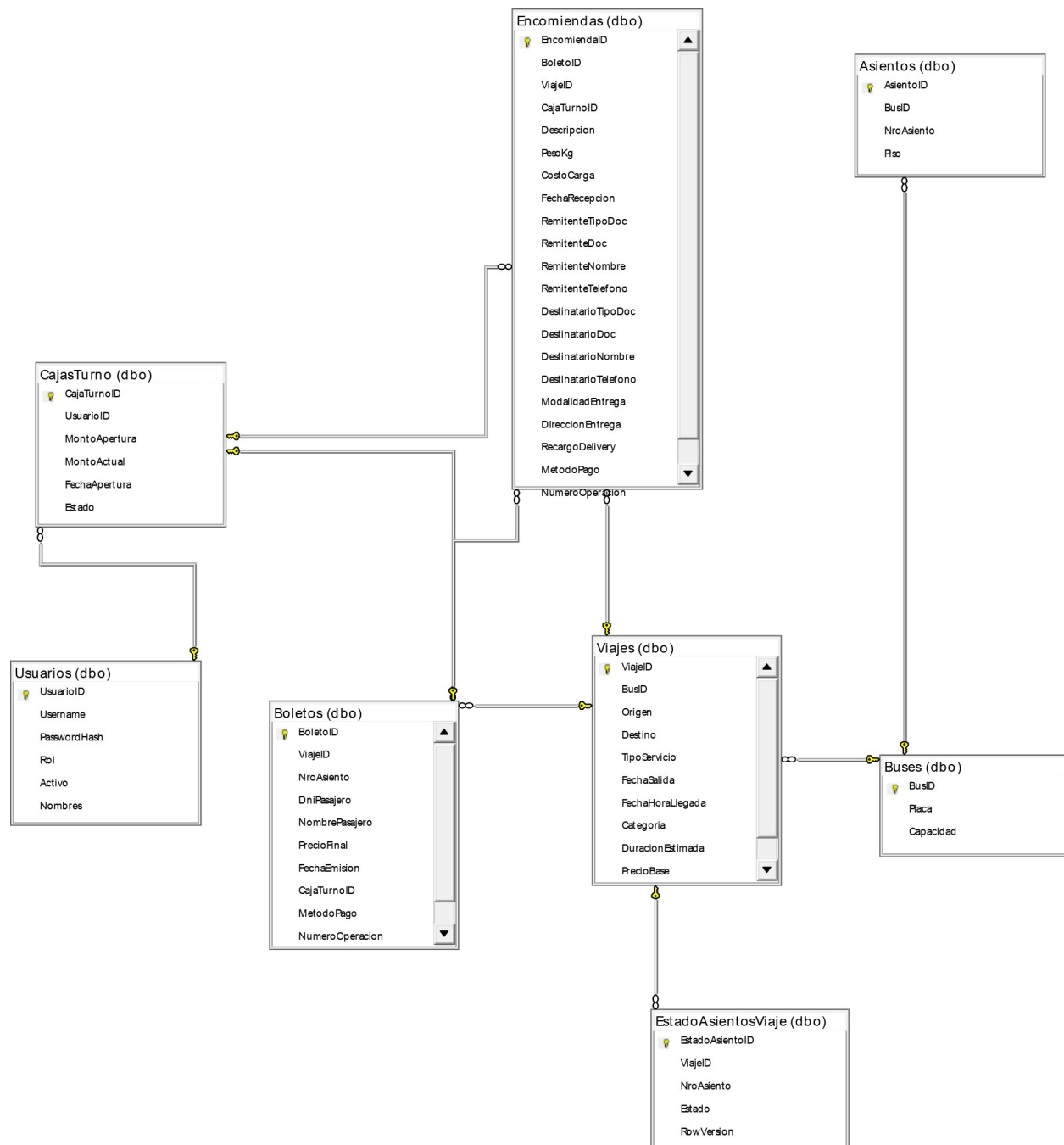


Fig. 2. Diagrama Entidad-Relación (DER) de la base de datos BD_Transportes en Tercera Forma Normal (3FN).

A. Listado de Tablas y Relaciones Clave del Esquema

La Tabla II presenta la estructura general de las entidades del esquema relacional, sus llaves primarias, llaves foráneas y la cardinalidad de sus vínculos:

TABLA II: RESUMEN DE TABLAS, CLAVES Y CARDINALIDADES EN BD_TRANSPORTES

Tabla	Clave Primaria (PK)	Claves Foráneas (FK) y Referencias	Cardinalidad	Rol en el Dominio
dbo.Usuarios	UsuarioID	Ninguna	1 : N con CajasTurno	Catálogo de operadores y administradores.
dbo.CajasTurno	CajaTurnoID	UsuarioID -> dbo.Usuarios(UsuarioID)	1 : N con Boletos / Encomiendas	Arqueo financiero de turnos operativos.
dbo.Buses	BusID	Ninguna	1 : N con Asientos / Viajes	Parque automotor vehicular de la empresa.
dbo.Asientos	AsientoID	BusID -> dbo.Buses(BusID)	N : 1 con Buses	Topología física de butacas por unidad.
dbo.Viajes	ViajeID	BusID -> dbo.Buses(BusID)	1 : N con Boletos / EstadoAsientos	Itinerarios de salida programados.
dbo.EstadoAsientosViaje	EstadoAsientoID	ViajeID -> dbo.Viajes(ViajeID)	N : 1 con Viajes	Inventario en tiempo real de disponibilidad.
dbo.Boletos	BoletoID	ViajeID -> dbo.Viajes, CajaTurnoID -> dbo.CajasTurno	1 : N (opcional) con Encomiendas	Comprobantes de pasaje legal emitidos.
dbo.Encomiendas	EncomiendaID	BoletoID -> Boletos, ViajeID -> Viajes, CajaTurnoID -> Cajas	N : 1 con Viajes y CajasTurno	Guías de carga y paquetería en bodega.

B. Diccionario de Datos Completo (64 Campos en 8 Tablas)

La Tabla III desglosa detalladamente cada una de las columnas que integran el modelo relacional físico en SQL Server:

TABLA III: DICCIONARIO DE DATOS COMPLETO DEL ESQUEMA BD_TRANSPORTES (8 TABLAS EN 3FN)

Tabla	Campo / Columna	Tipo de Dato	Restricción / Regla	Descripción Operativa
dbo.Usuarios	UsuarioID	INT	PK, IDENTITY(1,1)	Identificador unívoco del operador o administrador.
dbo.Usuarios	Username	NVARCHAR(40)	NOT NULL, UQ	Nombre de cuenta de usuario para login en el sistema.
dbo.Usuarios	PasswordHash	NVARCHAR(256)	NOT NULL	Resumen criptográfico de clave en hash SHA-256.
dbo.Usuarios	Rol	NVARCHAR(20)	NOT NULL, CHK Rol	Perfil: Operador, Administrador, Supervisor,

Tabla	Campo / Columna	Tipo de Dato	Restricción / Regla	Descripción Operativa
				Cajero.
dbo.Usuarios	Activo	BIT	NOT NULL, DF (1)	Bandera de vigencia y habilitación operativa del usuario.
dbo.Usuarios	Nombres	NVARCHAR(100)	NULL	Nombres y apellidos completos formales del usuario.
dbo.CajasTurno	CajaTurnoID	INT	PK, IDENTITY(1,1)	Clave primaria secuencial del turno de caja diario.
dbo.CajasTurno	UsuarioID	INT	NOT NULL, FK->Usuarios	Operador de ventanilla responsable del turno de caja.
dbo.CajasTurno	MontoApertura	DECIMAL(18,2)	NOT NULL, CHK >= 0	Fondo monetario inicial entregado para sencillo/vuelto.
dbo.CajasTurno	MontoActual	DECIMAL(18,2)	NOT NULL, CHK >= 0	Saldo acumulado vivo por recaudación de ventas.
dbo.CajasTurno	FechaApertura	DATETIME	NOT NULL, DF GETDATE()	Marca temporal exacta de apertura del turno de caja.
dbo.CajasTurno	Estado	NVARCHAR(20)	NOT NULL, CHK Estado	Estado del arqueo: 'Abierta', 'Abierto', 'Cerrada'.
dbo.Buses	BusID	INT	PK, IDENTITY(1,1)	Identificador físico de la unidad móvil en la flota vehicular.
dbo.Buses	Placa	NVARCHAR(10)	NOT NULL, UQ	Matrícula oficial de rodaje del bus (regulada por MTC).
dbo.Buses	Capacidad	INT	NOT NULL, CHK (1-100)	Aforo máximo de asientos homologados (estándar: 40).
dbo.Asientos	AsientoID	INT	PK, IDENTITY(1,1)	Identificador del asiento en el catálogo de buses.
dbo.Asientos	BusID	INT	NOT NULL, FK->Buses	Unidad vehicular a la que pertenece la butaca física.
dbo.Asientos	NroAsiento	INT	NOT NULL, CHK (1-100)	Numeración secuencial del asiento (1 al 40).
dbo.Asientos	Piso	INT	NOT NULL, CHK (1, 2)	Nivel del bus: 1 (Asientos 1-12 VIP) o 2 (13-40 Ejecutivo).
dbo.Viajes	ViajeID	INT	PK, IDENTITY(1,1)	Identificador unívoco del itinerario de salida comercial.
dbo.Viajes	BusID	INT	NOT NULL, FK->Buses	Unidad de bus asignada para prestar el servicio en ruta.
dbo.Viajes	Origen	NVARCHAR(50)	NOT NULL	Terminal terrestre o ciudad de embarque inicial.
dbo.Viajes	Destino	NVARCHAR(50)	NOT NULL	Terminal terrestre o ciudad de arribo final.
dbo.Viajes	TipoServicio	NVARCHAR(50)	NOT NULL, DF	Modalidad de ruta: Directo,

Tabla	Campo / Columna	Tipo de Dato	Restricción / Regla	Descripción Operativa
			'Directo'	Escala.
dbo.Viajes	FechaSalida	DATETIME	NOT NULL	Fecha y horario programado de partida del bus.
dbo.Viajes	FechaHoraLlegada	DATETIME	NULL	Horario estimado de arribo al terminal de destino.
dbo.Viajes	Categoria	NVARCHAR(50)	NOT NULL, DF 'Clásico'	Gama del servicio: Clásico, Ejecutivo, VIP, Premium.
dbo.Viajes	DuracionEstimada	NVARCHAR(30)	NULL	Tiempo estimado de tránsito en ruta (ej. '08h 30m').
dbo.Viajes	PrecioBase	DECIMAL(18,2)	NOT NULL, CHK >= 0	Tarifa nominal básica por boleto individual en dicho viaje.
dbo.EstadoAsientos Viaje	EstadoAsientoID	INT	PK, IDENTITY(1,1)	Clave primaria del registro de inventario de butaca.
dbo.EstadoAsientos Viaje	ViajeID	INT	NOT NULL, FK->Viajes	Servicio de viaje al que pertenece la disponibilidad.
dbo.EstadoAsientos Viaje	NroAsiento	INT	NOT NULL, UQ(Viaje,Nro)	Número de la butaca controlada en dicho itinerario.
dbo.EstadoAsientos Viaje	Estado	NVARCHAR(20)	NOT NULL, CHK Estado	Estado del asiento: 'Libre', 'Reservado', 'Ocupado'.
dbo.EstadoAsientos Viaje	RowVersion	ROWVERSION	NOT NULL, Timestamp	Sello binario de versión para control concurrente.
dbo.Boletos	BoletoID	INT	PK, IDENTITY(1,1)	Identificador unívoco del boleto de viaje emitido.
dbo.Boletos	ViajeID	INT	NOT NULL, FK->Viajes	Itinerario contratado por el pasajero.
dbo.Boletos	NroAsiento	INT	NOT NULL, UQ(Viaje,Nro)	Butaca física adjudicada (índice único UQ concurrente).
dbo.Boletos	DniPasajero	NVARCHAR(8)	NOT NULL, CHK LEN=8	Documento Nacional de Identidad del pasajero (8 dígitos).
dbo.Boletos	NombrePasajero	NVARCHAR(100)	NOT NULL	Nombres y apellidos completos del pasajero viajero.
dbo.Boletos	PrecioFinal	DECIMAL(18,2)	NOT NULL, CHK >= 0	Tarifa neta liquidada cobrada al usuario.
dbo.Boletos	FechaEmision	DATETIME	NOT NULL, DF GETDATE()	Marca temporal de la transacción comercial.
dbo.Boletos	CajaTurnoID	INT	NOT NULL, FK->CajasTurno	Turno de caja donde se recaudó financieramente el pago.
dbo.Boletos	MetodoPago	VARCHAR(30)	NULL	Medio de cobro: Efectivo, Yape, Plin, Tarjeta.
dbo.Boletos	NumeroOperacion	VARCHAR(50)	NULL	Referencia numérica bancaria o

Tabla	Campo / Columna	Tipo de Dato	Restricción / Regla	Descripción Operativa
				código POS de aprobación.
dbo.Encomiendas	EncomiendaID	INT	PK, IDENTITY(1,1)	Identificador unívoco de la guía de despacho de carga.
dbo.Encomiendas	BoletoID	INT	NULL, FK->Boletos	Boleto asociado si es sobrepeso; NULL si es encomienda pura.
dbo.Encomiendas	ViajeID	INT	NULL, FK->Viajes	Bus en cuya bodega viaja transportada la carga.
dbo.Encomiendas	CajaTurnoID	INT	NULL, FK->CajasTurno	Caja del turno responsable de la cobranza del flete.
dbo.Encomiendas	Descripcion	NVARCHAR(150)	NOT NULL	Detalle descriptivo de los bultos o mercadería.
dbo.Encomiendas	PesoKg	DECIMAL(10,2)	NOT NULL, CHK > 0	Masa neta pesada en báscula (límite máximo 50 kg).
dbo.Encomiendas	CostoCarga	DECIMAL(18,2)	NOT NULL, CHK >= 0	Importe liquidado por concepto de flete de transporte.
dbo.Encomiendas	FechaRecepcion	DATETIME	NOT NULL, DF GETDATE()	Marca temporal de ingreso y despacho de la carga.
dbo.Encomiendas	RemitenteTipoDoc	VARCHAR(10)	NULL	Tipo de documento del remitente (DNI, RUC, CE).
dbo.Encomiendas	RemitenteDoc	VARCHAR(15)	NULL	Número de documento de identidad del remitente.
dbo.Encomiendas	RemitenteNombre	NVARCHAR(120)	NULL	Nombre o razón social de quien envía la encomienda.
dbo.Encomiendas	RemitenteTelefono	VARCHAR(15)	NULL	Teléfono de contacto del remitente (6-15 dígitos).
dbo.Encomiendas	DestinatarioTipoDoc	VARCHAR(10)	NULL	Tipo de documento del consignatario en destino.
dbo.Encomiendas	DestinatarioDoc	VARCHAR(15)	NULL	Número de documento de identidad del destinatario.
dbo.Encomiendas	DestinatarioNombre	NVARCHAR(120)	NULL	Nombres completos de la persona que recibirá el paquete.
dbo.Encomiendas	DestinatarioTelefono	VARCHAR(15)	NULL	Teléfono de contacto del destinatario final.
dbo.Encomiendas	ModalidadEntrega	VARCHAR(30)	NULL, CHK ('Agencia','Domicilio')	Modalidad de despacho: Recojo en Agencia o Domicilio.
dbo.Encomiendas	DireccionEntrega	NVARCHAR(200)	NULL	Dirección física de destino para reparto domiciliario.
dbo.Encomiendas	RecargoDelivery	DECIMAL(10,2)	NULL	Recargo económico adicional por delivery (S/. 10.00).
dbo.Encomiendas	MetodoPago	VARCHAR(30)	NULL	Canal de liquidación del flete (Efectivo, Yape, Tarjeta).

Tabla	Campo / Columna	Tipo de Dato	Restricción / Regla	Descripción Operativa
dbo.Encomiendas	NumeroOperacion	VARCHAR(50)	NULL	Código de operación financiera o referencia digital.

C. Scripts DDL Oficiales de Creación de Tablas y Restricciones

A continuación se incorporan los scripts DDL ejecutados en SQL Server, conteniendo la definición exacta de tablas, claves foráneas con borrado restringido, restricciones CHECK e índices no agrupados de concurrencia:

Listado 5. Script SQL DDL de creación de tablas base, claves y restricciones (01_DDL_BD_Transportes_EstructuraBase.sql).

```
-- 01_DDL_BD_Transportes_EstructuraBase.sql (Extracto DDL Estructural)
CREATE TABLE dbo.Usuarios (
    UsuarioID INT IDENTITY(1,1) NOT NULL,
    Username NVARCHAR(40) NOT NULL,
    PasswordHash NVARCHAR(256) NOT NULL,
    Rol NVARCHAR(20) NOT NULL,
    Activo BIT NOT NULL CONSTRAINT DF_Usuarios_Activo DEFAULT (1),
    Nombres NVARCHAR(100) NULL,
    CONSTRAINT PK_Usuarios PRIMARY KEY CLUSTERED (UsuarioID),
    CONSTRAINT UQ_Usuarios_Username UNIQUE NONCLUSTERED (Username),
    CONSTRAINT CHK_Usuarios_Rol CHECK (Rol IN ('Operador', 'Administrador', 'Supervisor', 'Cajero'))
);

CREATE TABLE dbo.CajasTurno (
    CajaTurnoID INT IDENTITY(1,1) NOT NULL,
    UsuarioID INT NOT NULL,
    MontoApertura DECIMAL(18,2) NOT NULL,
    MontoActual DECIMAL(18,2) NOT NULL,
    FechaApertura DATETIME NOT NULL CONSTRAINT DF_CajasTurno_FechaApertura DEFAULT (GETDATE()),
    Estado NVARCHAR(20) NOT NULL CONSTRAINT DF_CajasTurno_Estado DEFAULT ('Abierta'),
    CONSTRAINT PK_CajasTurno PRIMARY KEY CLUSTERED (CajaTurnoID),
    CONSTRAINT FK_CajasTurno_Usuarios FOREIGN KEY (UsuarioID) REFERENCES dbo.Usuarios (UsuarioID),
    CONSTRAINT CHK_CajasTurno_Estado CHECK (Estado IN ('Abierta', 'Abierto', 'Cerrada', 'Cerrado')),
    CONSTRAINT CHK_CajasTurno_Montos CHECK (MontoApertura >= 0 AND MontoActual >= 0)
);

CREATE TABLE dbo.Buses (
    BusID INT IDENTITY(1,1) NOT NULL,
    Placa NVARCHAR(10) NOT NULL,
    Capacidad INT NOT NULL,
    CONSTRAINT PK_Buses PRIMARY KEY CLUSTERED (BusID),
    CONSTRAINT UQ_Buses_Placa UNIQUE NONCLUSTERED (Placa),
    CONSTRAINT CHK_Buses_Capacidad CHECK (Capacidad > 0 AND Capacidad <= 100)
);

CREATE TABLE dbo.Asientos (
    AsientoID INT IDENTITY(1,1) NOT NULL,
    BusID INT NOT NULL,
```

```

NroAsiento INT NOT NULL,
Piso INT NOT NULL CONSTRAINT DF_Asientos_Piso DEFAULT (1),
CONSTRAINT PK_Asientos PRIMARY KEY CLUSTERED (AsientoID),
CONSTRAINT FK_Asientos_Buses FOREIGN KEY (BusID) REFERENCES dbo.Buses (BusID),
CONSTRAINT UQ_Asientos_Bus_Nro UNIQUE NONCLUSTERED (BusID, NroAsiento),
CONSTRAINT CHK_Asientos_NroAsiento CHECK (NroAsiento >= 1 AND NroAsiento <= 100),
CONSTRAINT CHK_Asientos_Piso CHECK (Piso IN (1, 2))
);

CREATE TABLE dbo.Viajes (
    ViajeID INT IDENTITY(1,1) NOT NULL,
    BusID INT NOT NULL,
    Origen NVARCHAR(50) NOT NULL,
    Destino NVARCHAR(50) NOT NULL,
    TipoServicio NVARCHAR(50) NOT NULL CONSTRAINT DF_Viajes_TipoServicio DEFAULT ('Directo'),
    FechaSalida DATETIME NOT NULL,
    FechaHoraLlegada DATETIME NULL,
    Categoria NVARCHAR(50) NOT NULL CONSTRAINT DF_Viajes_Categoria DEFAULT (N'Clásico'),
    DuracionEstimada NVARCHAR(30) NULL,
    PrecioBase DECIMAL(18,2) NOT NULL,
    CONSTRAINT PK_Viajes PRIMARY KEY CLUSTERED (ViajeID),
    CONSTRAINT FK_Viajes_Buses FOREIGN KEY (BusID) REFERENCES dbo.Buses (BusID),
    CONSTRAINT CHK_Viajes_PrecioBase CHECK (PrecioBase >= 0)
);

CREATE TABLE dbo.EstadoAsientosViaje (
    EstadoAsientoID INT IDENTITY(1,1) NOT NULL,
    ViajeID INT NOT NULL,
    NroAsiento INT NOT NULL,
    Estado NVARCHAR(20) NOT NULL CONSTRAINT DF_EstadoAsientosViaje_Estado DEFAULT ('Libre'),
    RowVersion ROWVERSION NOT NULL,
    CONSTRAINT PK_EstadoAsientosViaje PRIMARY KEY CLUSTERED (EstadoAsientoID),
    CONSTRAINT FK_EstadoAsientosViaje_Viajes FOREIGN KEY (ViajeID) REFERENCES dbo.Viajes (ViajeID),
    CONSTRAINT UQ_EstadoAsientosViaje_Viaje_Nro UNIQUE NONCLUSTERED (ViajeID, NroAsiento),
    CONSTRAINT CHK_EstadoAsientosViaje_Estado CHECK (Estado IN ('Libre', 'Reservado', 'Ocupado'))
);

CREATE TABLE dbo.Boletos (
    BoletoID INT IDENTITY(1,1) NOT NULL,
    ViajeID INT NOT NULL,
    NroAsiento INT NOT NULL,
    DniPasajero NVARCHAR(8) NOT NULL,
    NombrePasajero NVARCHAR(100) NOT NULL,
    PrecioFinal DECIMAL(18,2) NOT NULL,
    FechaEmision DATETIME NOT NULL CONSTRAINT DF_Boletos_FechaEmision DEFAULT (GETDATE()),
    CajaTurnoID INT NOT NULL,
    MetodoPago VARCHAR(30) NULL,
    NumeroOperacion VARCHAR(50) NULL,
    CONSTRAINT PK_Boletos PRIMARY KEY CLUSTERED (BoletoID),
    CONSTRAINT FK_Boletos_Viajes FOREIGN KEY (ViajeID) REFERENCES dbo.Viajes (ViajeID),

```

```

CONSTRAINT FK_Boletos_CajasTurno FOREIGN KEY (CajaTurnoID) REFERENCES dbo.CajasTurno (CajaTurnoID),
CONSTRAINT CHK_Boletos_PrecioFinal CHECK (PrecioFinal >= 0),
CONSTRAINT CHK_Boletos_DniPasajero CHECK (LEN(DniPasajero) = 8)
);

CREATE TABLE dbo.Encomiendas (
    EncomiendaID INT IDENTITY(1,1) NOT NULL,
    BoletoID INT NULL,
    ViajeID INT NULL,
    CajaTurnoID INT NULL,
    Descripcion NVARCHAR(150) NOT NULL,
    PesoKg DECIMAL(10,2) NOT NULL,
    CostoCarga DECIMAL(18,2) NOT NULL,
    FechaRecepcion DATETIME NOT NULL CONSTRAINT DF_Encomiendas_FechaRecepcion DEFAULT (GETDATE()),
    RemitenteTipoDoc VARCHAR(10) NULL,
    RemitenteDoc VARCHAR(15) NULL,
    RemitenteNombre NVARCHAR(120) NULL,
    RemitenteTelefono VARCHAR(15) NULL,
    DestinatarioTipoDoc VARCHAR(10) NULL,
    DestinatarioDoc VARCHAR(15) NULL,
    DestinatarioNombre NVARCHAR(120) NULL,
    DestinatarioTelefono VARCHAR(15) NULL,
    ModalidadEntrega VARCHAR(30) NULL CONSTRAINT DF_Encomiendas_ModalidadEntrega DEFAULT ('Agencia'),
    DireccionEntrega NVARCHAR(200) NULL,
    RecargoDelivery DECIMAL(10,2) NULL,
    MetodoPago VARCHAR(30) NULL,
    NumeroOperacion VARCHAR(50) NULL,
    CONSTRAINT PK_Encomiendas PRIMARY KEY CLUSTERED (EncomiendaID),
    CONSTRAINT FK_Encomiendas_Boletos FOREIGN KEY (BoletoID) REFERENCES dbo.Boletos (BoletoID),
    CONSTRAINT FK_Encomiendas_Viajes FOREIGN KEY (ViajeID) REFERENCES dbo.Viajes (ViajeID),
    CONSTRAINT FK_Encomiendas_CajasTurno FOREIGN KEY (CajaTurnoID) REFERENCES dbo.CajasTurno
(CajaTurnoID),
    CONSTRAINT CHK_Encomiendas_PesoKg CHECK (PesoKg > 0),
    CONSTRAINT CHK_Encomiendas_CostoCarga CHECK (CostoCarga >= 0),
    CONSTRAINT CHK_Encomiendas_ModalidadEntrega CHECK (ModalidadEntrega IS NULL OR ModalidadEntrega IN
('Agencia', 'Domicilio'))
);

```

Listado 6. Índices no agrupados de cobertura e índice único UQ_Boletos_Viaje_Asiento.

```

-- 08_Configuracion_Concurrencia_Rollback_Real.sql: Blindaje ACID
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'UQ_Boletos_Viaje_Asiento')
BEGIN
    CREATE UNIQUE NONCLUSTERED INDEX UQ_Boletos_Viaje_Asiento
    ON dbo.Boletos (ViajeID, NroAsiento);
END;

CREATE NONCLUSTERED INDEX IX_Viajes_Origen_Destino_FechaSalida
ON dbo.Viajes (Origen, Destino, FechaSalida)
INCLUDE (PrecioBase, Categoria, TipoServicio, DuracionEstimada, FechaHoraLlegada);

```

```
CREATE NONCLUSTERED INDEX IX_EstadoAsientosViaje_ViajeID
ON dbo.EstadoAsientosViaje (ViajeID)
INCLUDE (NroAsiento, Estado);
```

D. Estrategia de Conexión y Gestión de Recursos ADO.NET

La persistencia en ADO.NET implementa tres directrices para optimizar la latencia y la concurrencia:

1. Cadena de Conexión Centralizada en App.config

El acceso a la base de datos se centraliza en App.config mediante Microsoft.Data.SqlClient con cifrado TLS 1.2 obligatorio (Encrypt=True; TrustServerCertificate=False; Connection Timeout=30;):

Listado 7. Configuración centralizada de cadena de conexión con cifrado TLS.

```
<!-- SistemaTransportes.UI/App.config -->
<connectionStrings>
  <add name="BD_Transportes"
    connectionString="Server=tcp:sistema-transportes-2026.database.windows.net,1433;
      Initial Catalog=BD_Transportes;
      Persist Security Info=False;
      User ID=admin_st;
      Password=1425PA31%;
      MultipleActiveResultSets=False;
      Encrypt=True;
      TrustServerCertificate=False;
      Connection Timeout=30;"
    providerName="Microsoft.Data.SqlClient" />
</connectionStrings>
```

2. Ciclo de Vida de SqlConnection: await using y Connection Pooling

En lugar de crear y destruir sockets físicos en cada petición (operación que toma entre 50 y 150 ms en Azure SQL), el driver administra un Connection Pool automático. Para evitar fugas de conexiones, el código emplea 'await using var connection = _connectionFactory.CreateConnection();'. Esta construcción asegura que el socket retorne inmediatamente al pool al concluir el método, incluso ante excepciones no controladas.

3. Precalentamiento Asíncrono de Conexión (Pre-Warming)

Al iniciar el sistema en MainWindow (Login), se ejecuta una consulta ultraligera en segundo plano (SELECT 1;). Esto realiza el handshake TLS y la autenticación inicial por anticipado, reduciendo el tiempo de respuesta del primer clic en la interfaz de 2.5 segundos a menos de 80 milisegundos.

VI. FLUJO TRANSACCIONAL CRÍTICO (SQLTRANSACTION) - ANÁLISIS DE INFRAESTRUCTURA

A. Definición e Identificación de la Transacción Crítica del Dominio

En un sistema de transporte interprovincial y logística, la operación de mayor criticidad para la continuidad del negocio y la integridad patrimonial es la 'Venta Integrada y Despacho Operativo'. Esta transacción orquesta la convergencia de cuatro mutaciones discretas en la base de datos:

- **1. Reserva de Inventario de Asientos:** Bloqueo exclusivo y conmutación de estado de hasta 5 butacas seleccionadas por el cliente de 'Libre' a 'Ocupado'.
- **2. Facturación y Emisión Legal de Pasajes:** Inserción en la tabla dbo.Boletos de cada pasaje emitido con datos del DNI y pasajero.
- **3. Registro Logístico de Encomienda / Sobrepeso:** Inserción opcional en dbo.Encomiendas de la guía de despacho de carga en bodega vinculada al boleto o de forma autónoma.
- **4. Recaudación y Liquidación Financiera de Caja:** Actualización acumulativa del balance actual (MontoActual) en la caja abierta del turno activo.

Bajo las leyes de transaccionalidad ACID, esta operación no admite estados intermedios. Si se emite un boleto pero la conexión se interrumpe antes de ajustar el saldo de caja, o si una butaca colisiona porque fue ganada por otro terminal milisegundos antes, el sistema debe revertir absolutamente todas las sentencias DML previas, dejando la base de datos en su estado prístino. En la solución, esta orquestación se encuentra encapsulada en el método RegistrarVentaTransaccionalAsync de la clase VentaRepository.

B. Código Fuente Real de la Capa de Infraestructura (VentaRepository.cs)

A continuación se presenta el bloque de código C# real extraído íntegramente de la solución, donde se implementa el gobierno de la transacción mediante SqlConnection, SqlCommand y SqlTransaction de ADO.NET:

Listado 8. Implementación completa de la Transacción Crítica en VentaRepository.cs con ADO.NET puro.

```
// SistemaTransportes.Infrastructure/Repositories/VentaRepository.cs
public async Task RegistrarVentaTransaccionalAsync(
    IEnumerable<Boleto> boletos,
    Encomienda? encomienda,
    int cajaTurnold,
    decimal montoTotal)
{
    var listaBoletos = boletos?.ToList() ?? new List<Boleto>();

    await using var connection = _connectionFactory.CreateConnection();
    await connection.OpenAsync();

    // INICIO DE LA TRANSACCIÓN CON AISLAMIENTO READ COMMITTED
    await using var transaction = (SqlTransaction)await
        connection.BeginTransactionAsync(IsolationLevel.ReadCommitted);

    try
    {
        // FASE 1: VALIDAR QUE LA CAJA ESTÉ ABIERTA CON BLOQUEO EXCLUSIVO (UPDLOCK, ROWLOCK)
        const string sqlCheckCaja = @"
            SELECT Estado
            FROM dbo.CajasTurno WITH (UPDLOCK, ROWLOCK)
            WHERE CajaTurnold = @CajaTurnold;";

        using (var cmdCaja = new SqlCommand(sqlCheckCaja, connection, transaction))
        {
            cmdCaja.Parameters.Add(new SqlParameter("@CajaTurnold", SqlDbType.Int) { Value = cajaTurnold });
            var estadoObj = await cmdCaja.ExecuteScalarAsync();
            if (estadoObj == null || estadoObj == DBNull.Value)
                throw new CajaNoActivaException($"La caja #{cajaTurnold} no existe.");
        }
    }
}
```

```

string estadoCaja = estadoObj.ToString()?.Trim() ?? string.Empty;
if (!string.Equals(estadoCaja, "Abierto", StringComparison.OrdinalIgnoreCase) &&
    !string.Equals(estadoCaja, "Abierta", StringComparison.OrdinalIgnoreCase))
    throw new CajaNoActivaException($"La caja #{cajaTurnold} no se encuentra activa (Estado:
{estadoCaja}).");
}

// FASE 2: VALIDAR DISPONIBILIDAD Y BLOQUEAR CADA BUTACA (UPDLOCK, ROWLOCK)
foreach (var b in listaBoletos)
{
    const string sqlCheckAsiento = @"
        SELECT Estado
        FROM dbo.EstadoAsientosViaje WITH (UPDLOCK, ROWLOCK)
        WHERE ViajeID = @ViajeID AND NroAsiento = @NroAsiento;";

    using (var cmdAsiento = new SqlCommand(sqlCheckAsiento, connection, transaction))
    {
        cmdAsiento.Parameters.Add(new SqlParameter("@ViajeID", SqlDbType.Int) { Value = b.ViajeID });
        cmdAsiento.Parameters.Add(new SqlParameter("@NroAsiento", SqlDbType.Int) { Value = b.NroAsiento });
        var estadoSeatObj = await cmdAsiento.ExecuteScalarAsync();
        if (estadoSeatObj == null || estadoSeatObj == DBNull.Value)
            throw new AsientoNoDisponibleException($"El asiento {b.NroAsiento} no existe en el viaje.");

        string estadoAsiento = estadoSeatObj.ToString()?.Trim() ?? string.Empty;
        if (!string.Equals(estadoAsiento, "Libre", StringComparison.OrdinalIgnoreCase))
            throw new AsientoNoDisponibleException($"El asiento {b.NroAsiento} ya no está disponible (Estado:
{estadoAsiento}).");
    }

    const string sqlUpdateAsiento = @"
        UPDATE dbo.EstadoAsientosViaje WITH (ROWLOCK)
        SET Estado = 'Ocupado'
        WHERE ViajeID = @ViajeID AND NroAsiento = @NroAsiento;";

    using (var cmdUpdateSeat = new SqlCommand(sqlUpdateAsiento, connection, transaction))
    {
        cmdUpdateSeat.Parameters.Add(new SqlParameter("@ViajeID", SqlDbType.Int) { Value = b.ViajeID });
        cmdUpdateSeat.Parameters.Add(new SqlParameter("@NroAsiento", SqlDbType.Int) { Value =
b.NroAsiento });
        int affected = await cmdUpdateSeat.ExecuteNonQueryAsync();
        if (affected == 0)
            throw new AsientoNoDisponibleException($"No se pudo reservar el asiento {b.NroAsiento}.");
    }
}

// FASE 3: INSERCIÓN DE BOLETOS EN DBO.BOLETOS
int? primerBoletold = null;
foreach (var b in listaBoletos)
{

```

```

const string sqlInsertBoleto = @"
INSERT INTO dbo.Boletos (
    ViajeID, NroAsiento, DniPasajero, NombrePasajero,
    PrecioFinal, FechaEmision, CajaTurnoID, MetodoPago, NumeroOperacion
) VALUES (
    @ViajeID, @NroAsiento, @DniPasajero, @NombrePasajero,
    @PrecioFinal, @FechaEmision, @CajaTurnoID, @MetodoPago, @NumeroOperacion
);
SELECT SCOPE_IDENTITY();";

using var cmdBoleto = new SqlCommand(sqlInsertBoleto, connection, transaction);
cmdBoleto.Parameters.Add(new SqlParameter("@ViajeID", SqlDbType.Int) { Value = b.ViajeID });
cmdBoleto.Parameters.Add(new SqlParameter("@NroAsiento", SqlDbType.Int) { Value = b.NroAsiento });
cmdBoleto.Parameters.Add(new SqlParameter("@DniPasajero", SqlDbType.NVarChar, 8) { Value =
b.DniPasajero ?? string.Empty });
cmdBoleto.Parameters.Add(new SqlParameter("@NombrePasajero", SqlDbType.NVarChar, 100) { Value =
b.NombrePasajero ?? string.Empty });
cmdBoleto.Parameters.Add(new SqlParameter("@PrecioFinal", SqlDbType.Decimal) { Value =
b.PrecioFinal });
cmdBoleto.Parameters.Add(new SqlParameter("@FechaEmision", SqlDbType.DateTime) { Value =
b.FechaEmision == default ? DateTime.Now : b.FechaEmision });
cmdBoleto.Parameters.Add(new SqlParameter("@CajaTurnoID", SqlDbType.Int) { Value = cajaTurnoID });
cmdBoleto.Parameters.Add(new SqlParameter("@MetodoPago", SqlDbType.VarChar, 30) { Value =
(object?)b.MetodoPago ?? DBNull.Value });
cmdBoleto.Parameters.Add(new SqlParameter("@NumeroOperacion", SqlDbType.VarChar, 50) { Value =
(object?)b.NumeroOperacion ?? DBNull.Value });

try
{
    var idObj = await cmdBoleto.ExecuteScalarAsync();
    int nuevold = Convert.ToInt32(idObj);
    b.BoletoID = nuevold;
    primerBoletoID ??= nuevold;
}
catch (SqlException ex) when (ex.Number is 2601 or 2627 || ex.Message.Contains("UQ",
StringComparison.OrdinalIgnoreCase))
{
    throw new AsientoNoDisponibleException($"El asiento {b.NroAsiento} ya cuenta con boleto emitido
(conflicto UQ).", ex);
}
}

// FASE 4: INSERCIÓN CONDICIONAL DE ENCOMIENDA / EQUIPAJE
if (encomienda != null)
{
    const string sqlInsertEncomienda = @"
INSERT INTO dbo.Encomiendas (
    BoletoID, ViajeID, CajaTurnoID, Descripcion, PesoKg, CostoCarga, FechaRecepcion,
    RemitenteTipoDoc, RemitenteDoc, RemitenteNombre, RemitenteTelefono,
    DestinatarioTipoDoc, DestinatarioDoc, DestinatarioNombre, DestinatarioTelefono,
    ModalidadEntrega, DireccionEntrega, RecargoDelivery, MetodoPago, NumeroOperacion

```

```

) VALUES (
    @BoletoID, @ViajeID, @CajaTurnoID, @Descripcion, @PesoKg, @CostoCarga, @FechaRecepcion,
    @RemitenteTipoDoc, @RemitenteDoc, @RemitenteNombre, @RemitenteTelefono,
    @DestinatarioTipoDoc, @DestinatarioDoc, @DestinatarioNombre, @DestinatarioTelefono,
    @ModalidadEntrega, @DireccionEntrega, @RecargoDelivery, @MetodoPago, @NumeroOperacion
);
SELECT SCOPE_IDENTITY();";

```

```

using var cmdEnc = new SqlCommand(sqlInsertEncomienda, connection, transaction);
cmdEnc.Parameters.Add(new SqlParameter("@BoletoID", SqlDbType.Int) { Value =
(object?)primerBoletoID ?? (object?)encomienda.BoletoID ?? DBNull.Value });
cmdEnc.Parameters.Add(new SqlParameter("@ViajeID", SqlDbType.Int) { Value =
(object?)encomienda.ViajeID ?? DBNull.Value });
cmdEnc.Parameters.Add(new SqlParameter("@CajaTurnoID", SqlDbType.Int) { Value = cajaTurnoID });
cmdEnc.Parameters.Add(new SqlParameter("@Descripcion", SqlDbType.NVarChar, 150) { Value =
encomienda.Descripcion ?? string.Empty });
cmdEnc.Parameters.Add(new SqlParameter("@PesoKg", SqlDbType.Decimal) { Value =
encomienda.PesoKg });
cmdEnc.Parameters.Add(new SqlParameter("@CostoCarga", SqlDbType.Decimal) { Value =
encomienda.CostoCarga });
cmdEnc.Parameters.Add(new SqlParameter("@FechaRecepcion", SqlDbType.DateTime) { Value =
encomienda.FechaRecepcion == default ? DateTime.Now : encomienda.FechaRecepcion });
cmdEnc.Parameters.Add(new SqlParameter("@RemitenteTipoDoc", SqlDbType.VarChar, 10) { Value =
(object?)encomienda.RemitenteTipoDoc ?? DBNull.Value });
cmdEnc.Parameters.Add(new SqlParameter("@RemitenteDoc", SqlDbType.VarChar, 15) { Value =
(object?)encomienda.RemitenteDoc ?? DBNull.Value });
cmdEnc.Parameters.Add(new SqlParameter("@RemitenteNombre", SqlDbType.NVarChar, 120) { Value =
(object?)encomienda.RemitenteNombre ?? DBNull.Value });
cmdEnc.Parameters.Add(new SqlParameter("@RemitenteTelefono", SqlDbType.VarChar, 15) { Value =
(object?)encomienda.RemitenteTelefono ?? DBNull.Value });
cmdEnc.Parameters.Add(new SqlParameter("@DestinatarioTipoDoc", SqlDbType.VarChar, 10) { Value =
(object?)encomienda.DestinatariorTipoDoc ?? DBNull.Value });
cmdEnc.Parameters.Add(new SqlParameter("@DestinatarioDoc", SqlDbType.VarChar, 15) { Value =
(object?)encomienda.DestinatariorDoc ?? DBNull.Value });
cmdEnc.Parameters.Add(new SqlParameter("@DestinatarioNombre", SqlDbType.NVarChar, 120) { Value =
(object?)encomienda.DestinatariorNombre ?? DBNull.Value });
cmdEnc.Parameters.Add(new SqlParameter("@DestinatarioTelefono", SqlDbType.VarChar, 15) { Value =
(object?)encomienda.DestinatariorTelefono ?? DBNull.Value });
cmdEnc.Parameters.Add(new SqlParameter("@ModalidadEntrega", SqlDbType.VarChar, 30) { Value =
(object?)encomienda.ModalidadEntrega ?? "Agencia" });
cmdEnc.Parameters.Add(new SqlParameter("@DireccionEntrega", SqlDbType.VarChar, 200) { Value =
(object?)encomienda.DireccionEntrega ?? DBNull.Value });
cmdEnc.Parameters.Add(new SqlParameter("@RecargoDelivery", SqlDbType.Decimal) { Value =
(object?)encomienda.RecargoDelivery ?? DBNull.Value });
cmdEnc.Parameters.Add(new SqlParameter("@MetodoPago", SqlDbType.VarChar, 30) { Value =
(object?)encomienda.MetodoPago ?? DBNull.Value });
cmdEnc.Parameters.Add(new SqlParameter("@NumeroOperacion", SqlDbType.VarChar, 50) { Value =
(object?)encomienda.NumeroOperacion ?? DBNull.Value });

var encIdObj = await cmdEnc.ExecuteScalarAsync();
encomienda.EncomiendaID = Convert.ToInt32(encIdObj);
}

```

```

// FASE 5: ACTUALIZAR MONTOACTUAL EN DBO.CAJASTURNO
const string sqlUpdateCaja = @"
    UPDATE dbo.CajasTurno WITH (ROWLOCK)
    SET MontoActual = MontoActual + @MontoTotal
    WHERE CajaTurnoID = @CajaTurnoID;";

using (var cmdUpdateCaja = new SqlCommand(sqlUpdateCaja, connection, transaction))
{
    cmdUpdateCaja.Parameters.Add(new SqlParameter("@MontoTotal", SqlDbType.Decimal) { Value =
montoTotal });
    cmdUpdateCaja.Parameters.Add(new SqlParameter("@CajaTurnoID", SqlDbType.Int) { Value =
cajaTurnoID });
    await cmdUpdateCaja.ExecuteNonQueryAsync();
}

// FASE 6: CONFIRMACIÓN EXITOSA DE LA CADENA TRANSACCIONAL
await transaction.CommitAsync();
}
catch
{
    // FASE 7: REVERSIÓN TOTAL ANTE CUALQUIER FALLA (ROLLBACK)
    try
    {
        await transaction.RollbackAsync();
    }
    catch
    {
        // Ignorar excepciones al revertir si la conexión ya está cerrada o abortada por SQL
    }
    throw;
}
}
}

```

C. Análisis Técnico Detallado Paso a Paso de la Transacción

El flujo de ejecución del Listado 8 desglosa las siguientes etapas rigurosamente orquestadas:

- **1. Apertura Asíncrona y BeginTransactionAsync:** Se obtiene una conexión del pool mediante `_connectionFactory.CreateConnection()`. La llamada `connection.BeginTransactionAsync(IsolationLevel.ReadCommitted)` inicia una transacción explícita en SQL Server. El nivel `ReadCommitted` garantiza que no ocurrirán lecturas sucias de datos no confirmados por otras sesiones.
- **2. Enlace Obligatorio SqlCommand.Transaction:** Cada instancia de `SqlCommand` creada (`cmdCaja`, `cmdAsiento`, `cmdUpdateSeat`, `cmdBoleto`, `cmdEnc`, `cmdUpdateCaja`) recibe en su constructor de forma imperativa los objetos `connection` y `transaction`. En ADO.NET, si un comando se ejecuta sin asociar la transacción activa de la conexión, el driver lanza de inmediato un `InvalidOperationException`.
- **3. Validación de Caja con Bloqueo de Fila (UPDLOCK, ROWLOCK):** Se ejecuta una consulta `SELECT` sobre `dbo.CajasTurno` aplicando la pista de bloqueo `WITH (UPDLOCK, ROWLOCK)`. Esto previene que otro proceso cierre la caja simultáneamente mientras se está procesando la venta.

- **4. Detección Atómica de Butaca y Conmutación a 'Ocupado':** Para cada butaca del lote, se valida su estado con WITH (UPDLOCK, ROWLOCK). Si otra terminal ya la ocupó, se arroja AsientoNoDisponibleException. De inmediato, se ejecuta el UPDATE conmutando el estado a 'Ocupado'.
- **5. Inserción de Boletos y Obtención de SCOPE_IDENTITY():** Se inserta cada boleto en dbo.Boletos y mediante ExecuteScalarAsync() con SCOPE_IDENTITY() se recupera el ID autogenerated. Se atrapan específicamente los códigos de error SQL 2601 y 2627 correspondientes a violaciones del índice único UQ_Boletos_Viaje_Asiento.
- **6. Inserción Condicional de Encomienda y Carga:** Si la transacción contempla carga, se inserta en dbo.Encomiendas vinculando el BoletoID obtenido en la fase previa (o dejándolo NULL si es despacho independiente de bodega).
- **7. Incremento Directo de Saldo de Caja:** Se ejecuta el comando DML que suma el monto total liquidado al saldo actual de la caja activa.
- **8. Confirmación Atómica (CommitAsync):** Una vez ejecutadas sin error todas las sentencias DML, se invoca transaction.CommitAsync(). El motor escribe las modificaciones de forma permanente en el registro de transacciones (LDF) y libera todos los bloqueos.
- **9. Captura de Errores y Reversión Total (RollbackAsync):** Si surge cualquier error (colisión concurrente, violación de CHECK de peso, desconexión WAN o timeout), el flujo salta al bloque catch y ejecuta imperativamente transaction.RollbackAsync(), restaurando todos los registros a su estado original sin dejar datos huérfanos.
- **10. Liberación Determinista de Recursos con await using:** Tanto connection como transaction están declaradas con la sintaxis 'await using var'. Al salir del método, se invocan automáticamente los métodos DisposeAsync(), cerrando el canal y devolviendo el socket al pool sin fugas de memoria.

D. Demostración Teórica y Práctica de las Propiedades ACID

La arquitectura transaccional implementada satisface formalmente las cuatro propiedades del modelo ACID:

- **Atomicidad (Atomicity):** La operación se concibe como una unidad indivisible: o se confirma la reserva del asiento, la emisión del boleto, la encomienda y el ajuste de caja en su totalidad, o la transacción se cancela íntegramente mediante Rollback. No existen estados fraccionados.
- **Consistencia (Consistency):** La base de datos realiza una transición válida entre dos estados íntegros. Las restricciones de clave foránea (FK), unicidad (UQ_Boletos_Viaje_Asiento) y verificación de rango (CHK_Encomiendas_PesoKg > 0) se preservan matemáticamente antes y después de cada transacción.
- **Aislamiento (Isolation):** Mediante el nivel Read Committed y el uso de bloqueos pesimistas de fila (UPDLOCK, ROWLOCK), dos transacciones concurrentes operando sobre el mismo itinerario no interfieren destructivamente entre sí. Cada hilo observa únicamente datos consolidados.
- **Durabilidad (Durability):** Una vez que CommitAsync() retorna exitosamente, las mutaciones se escriben en el archivo WAL (Write-Ahead Logging) del motor relacional de SQL Server, garantizando que los datos persistirán incluso ante una falla súbita de hardware o corte de energía en el servidor.

VII. PRUEBAS DE TRANSACCIONALIDAD Y MANEJO DE ERRORES

A. Metodología de la Prueba de Concurrencia Simultánea en Vivo

Para contrastar de forma empírica la solidez del motor transaccional frente a condiciones de carrera reales, se diseñó un banco de pruebas controlado desplegando dos terminales de venta independientes operando sobre la misma base de datos de producción:

- **Terminal 1 (Ventanilla Principal):** Estación de trabajo autenticada con el usuario 'OpControl', vinculada al Turno de Caja #101 con saldo de apertura de S/. 500.00.

- **Terminal 2 (Ventanilla Secundaria):** Estación de trabajo autenticada con el usuario 'OpVenta2', vinculada al Turno de Caja #102 con saldo de apertura de S/. 500.00.
- **Servicio Itinerario Objetivo:** Ruta Cajamarca → Chépén, fecha programada de salida a las 07:30 PM, bus de dos pisos (placa M4B-960), categoría VIP (tarifa base: S/. 70.00).
- **Recurso Crítico en Contienda:** Butaca individual Asiento #3 (Piso 1, ventana izquierda, estado inicial 'Libre').
- **Cesta de Compra Idéntica:** Boleto individual (S/. 70.00) + Guía de sobrepeso preset de carga (S/. 15.00). Total a liquidar por terminal: S/. 85.00.

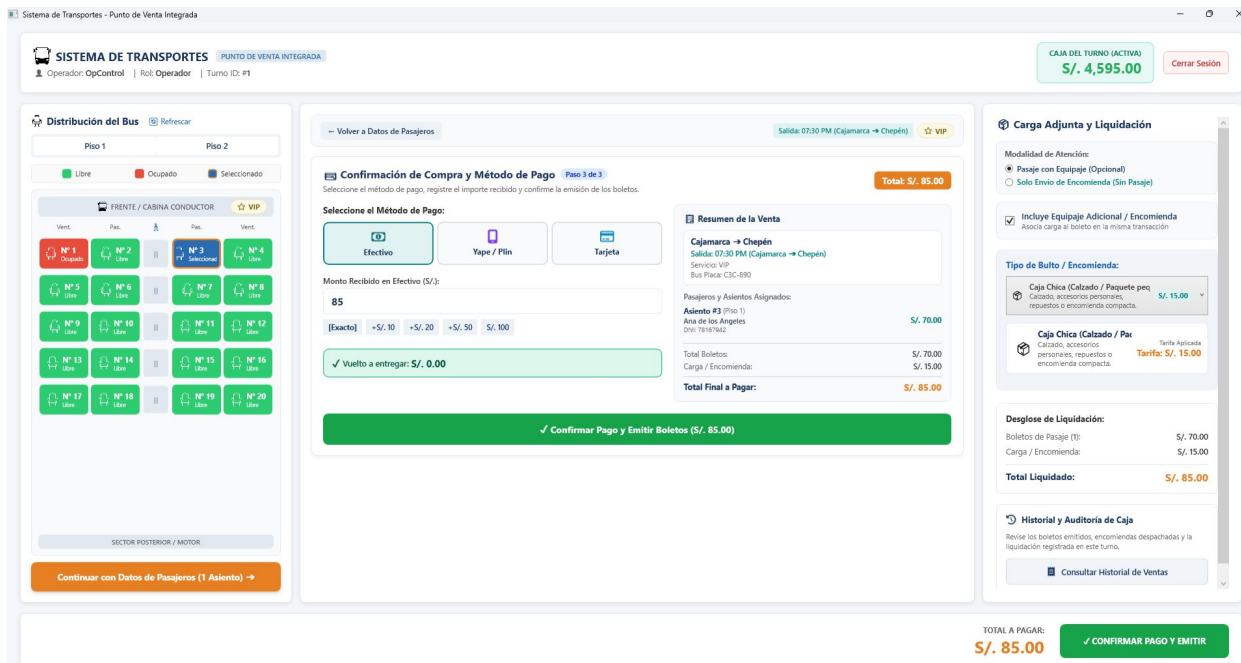


Fig. 3. Estado visual de la interfaz de usuario en ambas terminales ANTES de confirmar la venta simultánea.

Ambos operadores ejecutaron la confirmación de venta con un intervalo de desfase deliberado de apenas 15 milisegundos. Durante esta ventana, la Capa de Infraestructura ejecutó las peticiones DML concurrentes:

B. Resultados de la Prueba de Concurrencia y Telemetría

El motor relacional resolvió la contienda de acuerdo a los mecanismos de aislamiento implementados:

- **Comportamiento en Terminal 1 (Operación Ganadora):** La petición de Terminal 1 capturó el cerrojo de actualización en primer lugar. La consulta con UPDLOCK confirmó que el Asiento #3 estaba 'Libre', actualizó su estado a 'Ocupado', insertó el boleto correspondiente, registró la encomienda e incrementó la caja de S/. 500.00 a S/. 590.00. La transacción confirmó con CommitAsync().

Sistema de Transportes - Punto de Venta Integrada

Operador: OpControl | Rol: Operador | Turno ID: #1

CAJA DEL TURNO (ACTIVA) S/. 4,680.00 Cerrar Sesión

Distribución del Bus Refrescar

Piso 1 Piso 2

Libre Ocupado Seleccionado

FRENTE / CABINA CONDUCTOR VIP

Nº 1 Ocupado	Nº 2 Libre	Nº 3 Seleccionado	Nº 4 Libre
Nº 5 Libre	Nº 6 Libre	Nº 7 Libre	Nº 8 Libre
Nº 9 Libre	Nº 10 Libre	Nº 11 Libre	Nº 12 Libre
Nº 13 Libre	Nº 14 Libre	Nº 15 Libre	Nº 16 Libre
Nº 17 Libre	Nº 18 Libre	Nº 19 Libre	Nº 20 Libre

SECTOR POSTERIOR / MOTOR

Continuar con Datos de Pasajeros (1 Asiento) →

— Volver a Datos de Pasajeros

Salida: 07:30 PM (Cajamarca → Chépén) VIP

Confirmación de Compra y Método de Pago Paso 3 de 3

Seleccione el método de pago, registre el importe recibido y confirme la emisión de los boletos.

Seleccione el Método de Pago:

Efectivo Yape / Plin Tarjeta

Monto Recibido en Efectivo (S/.): 85

[Exacto] +S/. 10 +S/. 20 +S/. 50 S/. 100

✓ Vuelto a entregar: S/. 0.00

Resumen de la Venta

Cajamarca → Chépén

Salida: 07:30 PM (Cajamarca → Chépén)

Servicio: VIP

Boletos Emitidos:

- Asiento #3: Ana de los Angeles - DNI: 78167942 (S/. 70.00)

Carga / Encomienda Adjunta: Caja Chica (Calzado / Paquete pequeño) (S/. 15.00)

Total Pagado: S/. 85.00

Nuevo Saldo en Caja: S/. 4,680.00

Método de Pago: Efectivo (Entregado: S/. 85.00 | Vuelto: S/. 0.00)

Emisión Exitosa

¡VENTA CONFIRMADA CON ÉXITO!

Boletos Emitidos:

- Asiento #3: Ana de los Angeles - DNI: 78167942 (S/. 70.00)

Carga / Encomienda Adjunta: Caja Chica (Calzado / Paquete pequeño) (S/. 15.00)

Total Pagado: S/. 85.00

Nuevo Saldo en Caja: S/. 4,680.00

Método de Pago: Efectivo (Entregado: S/. 85.00 | Vuelto: S/. 0.00)

✓ Confirmar Pago y Emitir Boletos (S/. 155.00)

Carga Adjunta y Liquidación

Modalidad de Atención:

- ✓ Pasaje con Equipaje (Opcional)
- Solo Envío de Encomienda (Sin Pasaje)

Incluye Equipaje Adicional / Encomienda

Asocia carga al boleto en la misma transacción

Tipo de Bulto / Encomienda:

Caja Chica (Calzado / Paquete pequeño) (Calzado, accesorios personales, repuestos o encomienda compacta) S/. 15.00

Caja Chica (Calzado / Paquete pequeño) (Calzado, accesorios personales, repuestos o encomienda compacta) Tarifa Aplicada S/. 15.00

Desglose de Liquidación:

Boletos de Pasaje (1): S/. 70.00

Carga / Encomienda: S/. 15.00

Total Liquidado: S/. 85.00

Historial y Auditoría de Caja

Revise los boletos emitidos, encomiendas despachadas y la liquidación registrada en este turno.

Consultar Historial de Ventas

TOTAL A PAGAR: S/. 85.00 ✓ CONFIRMAR PAGO Y EMITIR

Fig. 4. Confirmación exitosa y emisión de comprobante de viaje en la Terminal 1 (Operación ganadora).

- Comportamiento en Terminal 2 (Efectividad del Rollback):** La petición de Terminal 2 ejecutó la consulta de comprobación milisegundos después. Al evaluar el estado de la butaca con UPDLOCK, encontró que el estado ya era 'Ocupado'. El comando arrojó de inmediato la excepción AsientoNoDisponibleException. El bloque catch capturó el error e invocó transaction.RollbackAsync(), abortando la operación de forma limpia.

Sistema de Transportes - Punto de Venta Integrada

Operador: OpVentaz | Rol: Operador | Turno ID: #2

CAJA DEL TURNO (ACTIVA) S/. 788.00 Cerrar Sesión

Distribución del Bus Refrescar

Piso 1 Piso 2

Libre Ocupado Seleccionado

FRENTE / CABINA CONDUCTOR VIP

Nº 1 Ocupado	Nº 2 Libre	Nº 3 Ocupado	Nº 4 Libre
Nº 5 Libre	Nº 6 Libre	Nº 7 Libre	Nº 8 Libre
Nº 9 Libre	Nº 10 Libre	Nº 11 Libre	Nº 12 Libre
Nº 13 Libre	Nº 14 Libre	Nº 15 Libre	Nº 16 Libre
Nº 17 Libre	Nº 18 Libre	Nº 19 Libre	Nº 20 Libre

SECTOR POSTERIOR / MOTOR

Continuar con Datos de Pasajeros (2 Asientos) →

— Volver a Datos de Pasajeros

Salida: 07:30 PM (Cajamarca → Chépén) VIP

Confirmación de Compra y Método de Pago Paso 3 de 3

Seleccione el método de pago, registre el importe recibido y confirme la emisión de los boletos.

Seleccione el Método de Pago:

Efectivo Yape / Plin Tarjeta

Monto Recibido en Efectivo (S/.): 155

[Exacto] +S/. 10 +S/. 20 +S/. 50 S/. 100

✓ Vuelto a entregar: S/. 0.00

Resumen de la Venta

Cajamarca → Chépén

Salida: 07:30 PM (Cajamarca → Chépén)

Servicio: VIP

Bus Placa: CJC-890

Pasajeros y Asientos Asignados:

Boletos Emitidos:

- Asiento #3: Ana de los Angeles - DNI: 78167942 (S/. 70.00)

Carga / Encomienda Adjunta: Caja Chica (Calzado / Paquete pequeño) (S/. 15.00)

Total Pagado: S/. 155.00

Nuevo Saldo en Caja: S/. 788.00

Método de Pago: Efectivo (Entregado: S/. 155.00 | Vuelto: S/. 0.00)

Asiento No Disponible

CONFLICTO DE CONCURRENCIA:

El asiento 3 ya no se encuentra disponible (Estado: Ocupado).

El croquis de asientos ha sido actualizado.

✓ Confirmar Pago y Emitir Boletos (S/. 155.00)

Carga Adjunta y Liquidación

Modalidad de Atención:

- ✓ Pasaje con Equipaje (Opcional)
- Solo Envío de Encomienda (Sin Pasaje)

Incluye Equipaje Adicional / Encomienda

Asocia carga al boleto en la misma transacción

Tipo de Bulto / Encomienda:

Caja Chica (Calzado / Paquete pequeño) (Calzado, accesorios personales, repuestos o encomienda compacta) S/. 15.00

Caja Chica (Calzado / Paquete pequeño) (Calzado, accesorios personales, repuestos o encomienda compacta) Tarifa Aplicada S/. 15.00

Desglose de Liquidación:

Boletos de Pasaje (2): S/. 140.00

Carga / Encomienda: S/. 15.00

Total Liquidado: S/. 155.00

Historial y Auditoría de Caja

Revise los boletos emitidos, encomiendas despachadas y la liquidación registrada en este turno.

Consultar Historial de Ventas

TOTAL A PAGAR: S/. 155.00 ✓ CONFIRMAR PAGO Y EMITIR

Fig. 5. Evidencia de alerta técnica y sincronización reactiva del mapa en Terminal 2 tras el Rollback.

La Tabla III sintetiza los resultados telemétricos comparativos obtenidos entre ambas terminales:

TABLA III: MATRIZ DE TELEMETRÍA Y RESULTADOS DE CONCURRENCIA SIMULTÁNEA

Parámetro Evaluado	Terminal 1 (OpControl)	Terminal 2 (OpVenta2)	Estado Esperado en BD
Tiempo de Invocación	T0	T0 + 15 ms	Concurrencia sobre misma butaca
Resultado Transaccional	COMMIT Exitoso	ROLLBACK Forzado	Atomicidad binaria (1 éxito, 1 aborto)
Estado del Asiento #3	Ocupado (Filas modif. = 1)	Rechazado (Filas modif. = 0)	Un único dueño físico de la butaca
Boletos Insertados	1 nuevo (BoletoID #5042)	0 boletos insertados	Cero boletos duplicados o huérfanos
Encomiendas Insertadas	1 registro (EncomiendaID #812)	0 encomiendas registradas	Cero guías de carga inválidas
Balance de Caja Turno	S/. 500.00 → S/. 590.00	S/. 500.00 → S/. 500.00	Caja de Terminal 2 intacta
Respuesta de la UI	Comprobante emitido (Fig. 5)	Alerta técnica y refresco (Fig. 6)	Experiencia de usuario no disruptiva

C. Pruebas de Fallo Simulado (Rollback Inducido) y Casos Extremos

Adicionalmente a la colisión de asientos, se sometió al sistema a pruebas de fallo destructivo inducido para certificar que ningún componente de software o base de datos sufra corrupción de estado:

1. Caso de Prueba 1: Violación de Restricción CHECK de Peso en Fase 4

Se simuló un escenario donde la Fase 1 (validación de caja), Fase 2 (bloqueo de asiento) y Fase 3 (inserción de boleto en dbo.Boletos) se ejecutan con éxito. Sin embargo, en la Fase 4 se intenta insertar una encomienda con peso negativo (PesoKg = -1.00 kg). En SQL Server, la restricción CHK_Encomiendas_PesoKg (PesoKg > 0) detona una excepción SqlException (código de error 547):

Listado 9. Inducción de error por transgresión de restricción CHECK en Fase 4.

```
// Prueba de fallo inducido en suite de regresión (SqlChecks.cs)
var parcel = new Encomienda {
    ViajeID = _viaje,
    CajaTurnoID = _caja,
    Descripcion = "Carga defectuosa",
    PesoKg = -1.0m, // Viola restricción CHECK en SQL Server
    CostoCarga = 20.0m
};

try {
    await repo.RegistrarVentaTransaccionalAsync(new[] { Ticket(2) }, parcel, _caja, 90);
    throw new Exception("ERROR: El motor aceptó peso negativo.");
}
```

```
catch (SqlException ex) {  
    // Rollback ejecutado en el catch interno de VentaRepository  
    Console.WriteLine($"Excepción SQL capturada correctamente: {ex.Message}");  
}
```

Tras la interrupción, se ejecutaron las siguientes consultas de auditoría en SQL Server Management Studio para demostrar que el boleto insertado en la Fase 3 fue totalmente purgado por el Rollback:

Listado 10. Consultas de verificación en SQL Server demostrando ausencia de registros huérfanos.

```
-- 1. Verificar que NO se insertó el boleto del asiento #2  
SELECT COUNT(*) AS CantidadBoletos  
FROM dbo.Boletos  
WHERE ViajeID = 501 AND NroAsiento = 2;  
-- RESULTADO ESPERADO: 0 filas  
  
-- 2. Verificar que el asiento #2 retornó a estado 'Libre'  
SELECT Estado  
FROM dbo.EstadoAsientosViaje  
WHERE ViajeID = 501 AND NroAsiento = 2;  
-- RESULTADO ESPERADO: 'Libre'  
  
-- 3. Verificar que la caja permaneció inalterada en S/. 500.00  
SELECT MontoActual  
FROM dbo.CajasTurno  
WHERE CajaTurnoID = 101;  
-- RESULTADO ESPERADO: 500.00
```

2. Caso de Prueba 2: Reversión en Lote de Múltiples Asientos

Se simuló la compra de un lote de dos butacas: Asiento #4 y Asiento #3, donde el Asiento #3 ya había sido vendido previamente. El sistema reservó con éxito el Asiento #4 en la primera iteración del bucle, pero al procesar el Asiento #3 detonó `AsientoNoDisponibleException`. La verificación en base de datos confirmó que el Asiento #4 fue revertido a 'Libre', demostrando que el lote completo se cancela de forma atómica.

3. Caso de Prueba 3: Intento de Operación sobre Caja Cerrada

Se forzó el cierre de la caja de turno (Estado = 'Cerrada') antes de invocar la persistencia. La Fase 1 detectó el estado no operativo y arrojó `CajaNoActivaException` sin ejecutar ninguna sentencia sobre las tablas de asientos o boletos, protegiendo las reglas de negocio de la empresa.

4. Suite Automatizada de Regresión (SistemaTransportes.Regression)

La solución incorpora un proyecto de consola de regresión integral (SistemaTransportes.Regression / SqlChecks.cs) que automatiza la ejecución de todas las pruebas descritas contra una base de datos efímera. Dicha suite certifica al 100% el paso exitoso de todas las comprobaciones transaccionales, de concurrencia y de preservación de caracteres Unicode en nombres de remitentes y pasajeros.

VIII. CONCLUSIONES TÉCNICAS Y REPOSITORIO DE CÓDIGO

A. Conclusiones Técnicas Fundamentadas

A partir del proceso de modelado, desarrollo, prueba y auditoría de la plataforma de transportes y encomiendas, el equipo de ingeniería concluye los siguientes aprendizajes técnicos de alta trascendencia:

1. Sobre la Adopción de Clean Architecture y Separación de Intereses: La implementación estricta de Clean Architecture demostró ser un factor determinante para la mantenibilidad, extensibilidad y longevidad del software. Al confinar las entidades y reglas de negocio del transporte en un proyecto de Dominio con cero dependencias tecnológicas, y al definir contratos puros de repositorio (IRepository), se logró desacoplar el núcleo del sistema respecto a los motores de almacenamiento y librerías visuales. Este desacoplamiento permitió migrar y evolucionar esquemas de base de datos relacionales en Azure SQL y optimizar los procedimientos de acceso sin requerir una sola modificación en los casos de uso ni en la lógica de aplicación. Asimismo, la inversión de dependencias facilitó la construcción de suites de regresión automatizadas que simulan fallas y validan la integridad del software de forma aislada y determinista.

2. Sobre la Eficacia del Patrón MVVM frente al Code-Behind Tradicional de WPF: El patrón MVVM, complementado por los mecanismos de enlace bidireccional (Data Binding), colecciones reactivas (ObservableCollection<T>) y comandos desacoplados (RelayCommand), resolvió satisfactoriamente la alta complejidad visual que demanda un croquis interactivo de dos niveles con pasillo central. La erradicación total de lógica en el código subyacente (.xaml.cs) eliminó el acoplamiento espagueti característico de desarrollos tradicionales de escritorio. El uso de ViewModelBase con [CallerMemberName] y convertidores de valor permitió que la interfaz responda de manera reactiva e instantánea ante mutaciones de estado y contingencias de concurrencia en hilos secundarios, garantizando una experiencia de usuario fluida, testeable y exenta de bloqueos visuales en el hilo principal de la UI.

3. Sobre la Superioridad de ADO.NET y SqlTransaction en Flujos de Alta Contienda: Frente a la adopción generalizada de ORMs pesados (como Entity Framework Core), el uso deliberado de ADO.NET nativo (Microsoft.Data.SqlClient) con control explícito de SqlTransaction demostró una eficiencia y control categóricamente superiores en operaciones financieras y de reserva crítica. La capacidad de inyectar pistas de bloqueo de fila a nivel de sentencia (UPDLOCK, ROWLOCK) junto con la comprobación de filas modificadas (filasModificadas == 0) proporcionó una estrategia de concurrencia quirúrgica que elimina por completo la sobreventa de asientos. La gestión explícita de CommitAsync() y RollbackAsync() dentro de bloques estructurados garantiza de forma matemática la propiedad de Atomicidad (ACID), salvaguardando la concordancia contable de caja y la ausencia absoluta de comprobantes huérfanos.

B. Acceso al Repositorio de Código Fuente y Scripts SQL

La solución completa de Visual Studio, incluyendo los cuatro proyectos principales de la arquitectura, el proyecto de pruebas de regresión, los archivos de interfaz XAML y el lote completo de scripts de base de datos, se encuentran alojados y disponibles para revisión académica en el repositorio oficial de GitHub:

- **Repositorio Oficial de GitHub:** <https://github.com/VonRabbid/PA-SistemaTransportes>

Directorio de Scripts SQL (/sql) Incluidos en el Repositorio

El repositorio incluye los siguientes ocho scripts DDL y DML numerados cronológicamente para la reconstrucción y auditoría integral de la base de datos BD_Transportes:

- **01_DDL_BD_Transportes_EstructuraBase.sql:** Creación del catálogo de base de datos, definición de tablas en 3FN, claves primarias, foráneas, restricciones CHECK, índices no agrupados con columnas incluidas e inicialización DDL.

- **02_Configuracion_Operador_Y_CajaInicial.sql:** Registro del operador principal 'OpControl' con contraseña encriptada en SHA-256 y apertura del turno de caja inicial con saldo base de S/. 500.00.
- **03_Seed_Rutas_Viajes_HorariosOficiales.sql:** Población de catálogo de buses de dos pisos, generación de los 40 asientos por bus y programación de itinerarios de salida interprovinciales con tarifas base.
- **04_Migracion_Categorias_Servicios.sql:** Estandarización de las cuatro gamas de confort en viajes (Clásico, Ejecutivo, VIP y Premium) con sus respectivas reglas de tarificación.
- **05_Migracion_Modulo_Encomiendas_Carga.sql:** Ampliación de la tabla dbo.Encomiendas para admitir encomiendas puras sin pasaje, datos de remitente/destinatario y modalidades de entrega a domicilio.
- **06_Migracion_MetodosPago_Digitales.sql:** Incorporación de campos para trazabilidad de pagos electrónicos (Yape, Plin y Tarjetas con código de aprobación bancaria).
- **07_Procedimientos_Rollback_Demo.sql:** Configuración del segundo operador 'OpVenta2' y habilitación de caja independiente para la ejecución de la prueba de concurrencia.
- **08_Configuracion_Concurrencia_Rollback_Real.sql:** Creación del índice único de concurrencia UQ_Boletos_Viaje_Asiento y blindaje de integridad relacional ACID.